

Plano do ERP Patria — v1

ERP grátis, multi-tenant SaaS e on-premise, monetizado via IA hipercomplexa de recomendação a fornecedores

Aarão Melo Lopes

Claude (Patria Technology)

25 de maio de 2026

Resumo executivo. Construir um ERP completo (PDV, estoque, financeiro, ordem de serviço, checkout) distribuído em dois modos a partir do mesmo código: SaaS multi-tenant em *.patriatechnology.com e instalação on-premise via Docker Compose na intranet do cliente. O produto é *grátis* para o lojista. Checkout entra 100% funcional desde o dia 1 via *conta interna* (Patria custodia o saldo em sua conta PagBank principal e faz payout para a conta bancária do lojista; default D+1 com opção de o lojista mudar para saque sob demanda) e oferece chave PagBank própria como opt-in. Taxa por transação repassada ao lojista sem markup. A receita vem da camada de IA: o NCO-HiperGNN (papers AF/AE/CT) ranqueia e sugere produtos, e o fornecedor/distribuidor paga pela visibilidade qualificada e por dados agregados anonimizados. Lançamento direto em escala nacional, sem cold start regional.

Sumário

1	Contexto e mercado-alvo	3
1.1	Público-alvo	3
1.2	Diagnóstico do mercado	3
1.3	Por que agora	3
2	Modelo de negócio	3
2.1	Princípio	3
2.2	Fontes de receita	3
2.3	Por que isso é defensável	4
3	Diferenciais técnicos derivados do NCO	4
3.1	O que o NCO-HiperGNN entrega ao ERP	4
3.2	Bound rigoroso vira SLA contratual	5
3.3	Decomposição micro/macro (paper CT)	5
4	Arquitetura	5
4.1	Topologia: um código, dois ambientes	5
4.2	Estrutura de diretório	6
4.3	Princípios de design herdados do integrator Easysync	6
4.4	Stack tecnológica	7
5	Módulos de domínio	7
5.1	Lista canônica	7
5.2	Modelo de dados resumido (núcleo)	8

6	Checkout: conta interna + chave própria opcional	9
6.1	Princípio: fricção zero	9
6.2	Base técnica reaproveitada da Patria	9
6.3	Três modos de operação por tenant	9
6.4	Ledger interno (modo <code>INTERNAL_WALLET</code>)	10
6.5	Política de payout (modelo híbrido)	10
6.6	Taxa: repasse sem markup	11
6.7	Fluxos suportados no PDV	11
6.8	Considerações on-premise	11
6.9	Migração futura para split nativo (PagBank Plus/Connect)	11
7	Sync cloud opt-in	12
7.1	Motivação	12
7.2	Modelo do que sobe	12
7.3	Modelo do que desce	12
7.4	Privacidade	12
8	Fases de entrega	12
8.1	F1 — Núcleo comum (auth + cadastros + estoque)	12
8.2	F2 — Vendas + Checkout + Caixa	13
8.3	F3 — Vertical OS (peças + serviços)	14
8.4	F4 — IA: NCO + sync de catálogo	14
8.5	F5 — Fiscal (NF-e/NFC-e/NFS-e)	14
8.6	F6 — Extras	15
9	Riscos e mitigações	15
10	Próximos passos imediatos	16

1 Contexto e mercado-alvo

1.1 Público-alvo

Quatro segmentos, com prioridade comercial nesta ordem:

1. **Comércio e materiais de construção** (lojas pequenas e médias). Mais carente de ERP decente, ticket médio razoável, baixa concorrência forte em N/NE.
2. **Peças e serviços** (autopeças, oficina, eletrônica, marcenaria). Combina produto físico com mão de obra; OS é central.
3. **Food service** (restaurantes, bares, lanchonetes), público da Pigz. Entrada posterior, com vantagem de já ter base de lojas instalada.
4. **Eventos** (casas noturnas, festas, festivais), também mira adjacente à Pigz; ingressos e comanda eletrônica.

1.2 Diagnóstico do mercado

O mercado de ERP brasileiro para PMEs é saturado em mensalidade fixa (Bling, Omie, Tiny, Conta Azul, TOTVS de pequeno porte) e em PDV-via-maquinhinha (Stone Ton, PagSeguro Moderninha, InfinitePay). Arquitetura, qualidade de UX ou catálogo de features *não* são diferenciadores defensáveis — todos os players conseguem construir tudo o que descrevemos como núcleo.

A brecha real está em **modelo de receita** e em **inteligência preditiva matematicamente fundada**, não em features.

1.3 Por que agora

A Patria Technology já tem operando três peças que ninguém na concorrência combina:

1. Infraestrutura SaaS multi-tenant em produção (Patria + Patrícia), roteamento por subdomínio, worker headless via Claude Code, deploy automático.
2. Solver neural NCO-HiperGNN treinado e validado (paper AF), com *bound de tempo* ($\sim 10\text{ms}$) e *bound de qualidade* ($C(s)^2/m^5$ rigoroso, paper AE) — combinação que não existe em outro solver disponível no mercado.
3. Integração PagBank em produção (PIX + cartão tokenizado), tabela `billing_events`, webhook em `nco.patriatechnology.com`, validada e aprovada em 13/05/2026.

2 Modelo de negócio

2.1 Princípio

Tudo grátis para o lojista, inclusive financeiro. Receita vem do lado do fornecedor, mediada por inteligência preditiva.

2.2 Fontes de receita

Cinco vetores, do mais simples ao mais complexo:

Vetor	Como cobra	Pré-requisito
Sugestão patrocinada de reposição	“Tá acabando seu cimento — sugiro Votorantim com 7% off agora.” Fornecedor paga por sugestão aceita (CPA) ou por lead qualificado (CPL). Sempre com <i>flag patrocinado</i> visível ao lojista.	Catálogo unificado de SKU + sinal de intenção de reposição (NCO forecast).
Marketplace de compras integrado	Lojista compra fornecedor dentro do ERP. Take rate (1–3%) sobre GMV, ou spread no crédito (parcelamento da compra).	Conexão B2B com distribuidores; capital de giro para antecipar (já disponível via fintech parceira).
Insights agregados para indústria	Dashboard tipo Nielsen para Coca, Brahma, Votorantim, indústrias de material e peças. “Marca X cresceu 12% no varejo de bairro de Boa Vista nas últimas 4 semanas.” Assinatura mensal recorrente.	Massa crítica de lojas (alguns milhares), sync agregado anonimizado opt-in.
Trade marketing digital ao consumidor	Banner/cupom no cardápio digital, totem, link de delivery do lojista. Marca paga CPM/CPC.	Canal de consumidor final implementado (food/eventos).
Crédito patrocinado pelo fornecedor	“Coca te empresta R\$ 5k a juro zero para repor estoque dela.” Patria intermedeia, leva spread.	Score de risco com base no histórico do ERP.

Fase 1 de receita: foco em (1) e (2), os mais rápidos de validar. (3) e (4) são consequência de massa instalada. (5) é prêmio depois da maturidade.

2.3 Por que isso é defensável

- **Versus ERPs tradicionais (Bling, Omie, Tiny, Conta Azul):** eles cobram mensalidade do lojista. Nós não cobramos. Quem está com caixa apertado nunca vai escolher pagar R\$ 200/mês se “grátis” existe.
- **Versus fintechs com PDV (Stone, PagSeguro, InfinitePay):** elas vieram da maquininha; o ERP delas é fraco ou adendo. Nós viemos do ERP, que é o cérebro da loja. ERP é *sticky*, maquininha é *commodity*.
- **Versus Pigz:** a Pigz cobra mensalidade. Nós não. E nossa IA de recomendação tem fundação matemática rigorosa (bound de qualidade), não treino-e-funciona.
- **Versus qualquer recomendador clássico:** NCO-HiperGNN tem bound rigoroso de tempo e qualidade. É o único quadrante marcado em [1]. Vira SLA contratual com fornecedor.

3 Diferenciais técnicos derivados do NCO

3.1 O que o NCO-HiperGNN entrega ao ERP

A camada de IA, executada localmente (CPU) ou via cloud (GPU em GEX44), resolve quatro problemas industriais com latência $\sim 10\text{ms}$ e variância $7,5\times$ menor que heurísticas clássicas:

1. **Forecast de venda por SKU.** Previsão de quantidade vendida no próximo período, derivada do histórico do ERP. Usa estrutura de cluster temporal $\rightarrow$ atribuição via partição.

2. **Detecção de ruptura de estoque.** Probabilidade de zerar dada a curva de venda + lead time do fornecedor.
3. **Segmentação automática de SKUs e clientes.** Grupos coesos para ações de venda (cross-sell, upsell), por simetria S_3 entre canais imaginários do quaternion.
4. **Roteamento e escalonamento.** Coloração de grafo para agenda de entregas, OS, alocação de mesa (food).

3.2 Bound rigoroso vira SLA contratual

Pela equação $C(s)^2 = 36(1 - s^2)^2$ (paper AE) e arquitetura $m = 4, L = 4$, o modelo entrega:

- Latência $p_{99} < 50\text{ms}$ para $n \leq 500$ SKUs.
- Variância de output $\sigma_K \leq C(s)/\sqrt{Km^5} \approx 0,07$ para $K = 8$ rollouts.
- Reprodutibilidade determinística com mesmo input + seed.

Esses três números viram cláusula de SLA para o cliente fornecedor que patrocina sugestões.

3.3 Decomposição micro/macro (paper CT)

A teoria de campo médio descrita no paper CT permite servir dois lados da plataforma de forma matematicamente coerente:

- **Micro — ao lojista.** Sugestão individual baseada na microdinâmica de cada elétron algébrico (paper A): “compre Y, promova Z, repor X agora”.
- **Macro — ao fornecedor.** Estado físico do agregado $S = \text{mean}_k(s_k v_k)$ sobre toda a rede de lojas, com decomposição $S = s_{\parallel} V + S_{\perp}$. Defeito perpendicular $S_{\perp}$ revela *heterogeneidade real* do mercado — insight que Nielsen e GfK não entregam com essa granularidade.

São *dois produtos sobre a mesma infraestrutura*, separáveis sem perder coerência.

4 Arquitetura

4.1 Topologia: um código, dois ambientes

Aspecto	SaaS	On-premise
Hosting	Servidor Patria (<code>erp.patriatechnology.com</code> , subdomínios)	Intranet do cliente (<code>erp.empresa.local</code>)
Auth	Login web Patria + plano	Login interno do cliente
Multi-tenant	Sim, via header X-Tenant	Sim (uma instalação serve N lojas da mesma empresa), X-Tenant interno; default <code>tenant=default</code> se single-loja
Banco	Postgres central com pgvector	Postgres local no Docker Compose
LLM	Anthropic via Patria	BYOK (Anthropic/OpenAI key do cliente) ou Ollama local opcional
NCO inference	Cloud (GEX44 ou réplica)	Container Python local com modelo TorchScript step 80k empacotado (CPU-only acceptable)

Aspecto	SaaS	On-premise
Cloud sync	N/A	Opt-in. Sobe agregados anonimizados, baixa catálogo de fornecedor e atualizações
Atualização	CI/CD automático	Manual via <code>docker compose pull && up -d</code>

Diferença vive em variáveis de ambiente e em uma tabela `Installation` com `mode`, `edition`, `cloudSyncEnabled`, `ncoMode`, `llmProvider`. Schema único.

4.2 Estrutura de diretório

```
patria-erp/
|-- erp-api/           NestJS + Prisma (já criado)
|-- erp-app/           Vite + React + TS (já criado)
|-- erp-worker/        Prisma direto, Claude headless, jobs
|-- nco-inference/     Python + FastAPI, TorchScript step 80k
|-- catalog-cloud/     Catálogo unificado de fornecedor (apenas SaaS)
|-- docker-compose.yml SaaS deployment
|-- docker-compose.local.yml on-premise bundle
|-- install.sh         instalador on-premise
|-- estrategia/        este plano e futuros docs
```

4.3 Princípios de design herdados do integrator Easysync

A leitura do `CLAUDE.md` do Easysync integrator (`/easysync/integrator/integrator/`) revela um ERP de varejo maduro (Spring Boot, DB2, Liquibase, Kafka+Debezium) com domínio bem modelado. Herdamos:

- Divisão modular: `product`, `customer-supplier`, `stock`, `warehouse`, `service-order`, `budget`, `order`, `payment`, `cash`, `invoice`, `promotion`, `vehicle`, `warranty-term`, `notification`, `webhook`.
- Entidade `CustomerSupplier` unificada (cliente e fornecedor no mesmo cadastro — decisão deles, mantida).
- Desenho do módulo `Cash`: `CashSession` com estados `OPEN/PARTIALLY_CLOSED/CLOSED`, `CashOperation` para sangria/reforço/troca de operador, `CashPhysicalClose` por forma de pagamento.
- Filtros dinâmicos via `SearchFilter` (no NestJS, helper Prisma-where).
- Feature flags por módulo (`*_ENABLED`).

Modernizamos:

Integrator (referência)	ERP Patria (nosso)
Spring Boot 3.3 / Java 21	NestJS 11 / TypeScript / Node 22
DB2 + Postgres secundário	Postgres único com pgvector
TF-IDF embeddings em <code>*_EMBEDDING</code>	pgvector vector(768) + Gemini embeddings, padrão Patria
Kafka + Debezium CDC	Outbox pattern interno + webhook HMAC, mais simples para escala média
Chatbot TF-IDF + GPT	Patrícia (Claude + tools) + NCO-HiperGNN com bound rigoroso

Integrator (referência)	ERP Patria (nosso)
JWT manual com jjwt	@nestjs/jwt + Passport
iText (PDF) + Apache POI (Excel) + escpos-coffee	V2; MVP usa puppeteer (PDF) + CSV; impressora térmica via driver web
GraalVM nativo opcional	NestJS standard (startup já rápido)

4.4 Stack tecnológica

- **Backend:** NestJS 11, TypeScript, Prisma 6, PostgreSQL 16 + pgvector, JWT via @nestjs/jwt, ConfigModule global.
- **Frontend:** Vite 5, React 19, TypeScript 5.6, proxy /api → :3001.
- **Worker:** Node 22, Prisma direto no banco, Claude Code headless via CLI (padrão patria-worker).
- **NCO inference:** Python 3.11, FastAPI, PyTorch (TorchScript congelado do checkpoint step 80k), CPU como default no on-premise.
- **Embeddings:** Gemini gemini-embedding-001 768d no SaaS; paraphrase-multilingual-mpnet-base-v2 local no on-premise (mesmo padrão do theory-embedder.service:9301 da Patria).
- **IA conversacional:** Claude Sonnet 4.6 (SaaS), Anthropic BYOK ou Ollama local (on-premise).
- **Pagamento:** integração PagBank reaproveitada da Patria (PIX + cartão tokenizado, tokenização client-side).
- **Deploy:** Docker Compose, imagens versionadas no registry.

5 Módulos de domínio

5.1 Lista canônica

Módulo	Escopo	Status
auth	login, JWT, roles, recuperação de senha	F1
installation	singleton com modo, edição, feature flags	F1
tenant	multi-tenant via X-Tenant, ALS interceptor	F1
customer-supplier	cadastro unificado cliente/fornecedor	F1
product	catálogo, hierarquia (Group/Subgroup/Section/Division), embeddings pgvector	F1
warehouse	depósitos	F1
stock	saldo por SKU/depósito, movimento, ajuste	F1
promotion + price	tabelas de preço, promoções	F2
order	venda, item, desconto, parcela	F2

Módulo	Escopo	Status
payment	métodos de pagamento, conciliação	F2
checkout	integração PagBank (PIX + cartão), tokenização client-side, webhook	F2
cash	sessão, sangria, reforço, troca de operador, fechamento físico	F2
service-order	OS, vinculação a veículo/equipamento, peça + mão de obra	F3
vehicle	veículo do cliente (oficina)	F3
warranty-term	termos de garantia	F3
budget	orçamento → efetiva em OS ou Order	F3
nco	forecast, ruptura, cluster, sugestão	F4
catalog-sync	sync opt-in cloud, ofertas patrocinadas	F4
invoice	NF-e/NFC-e/NFS-e, integração SEFAZ	F5
webhook	webhooks externos, HMAC, retry com backoff	F6
notification	websocket, in-app, e-mail	F6
logistic	frete, entrega	F6

5.2 Modelo de dados resumido (núcleo)

```

Installation { id, mode (saas|on_premise), edition, flags, createdAt }
Tenant       { id, alias, name, status, plan, createdAt }
User         { id, tenantId, email, hash, role, occupationId }

CustomerSupplier { id, tenantId, type (CUSTOMER|SUPPLIER|BOTH),
                    name, document, contacts, address, embedding }

Product      { id, tenantId, sku, name, division, section, group, subgroup,
                unit, brand, gtin, costPrice, embedding, ... }
Warehouse    { id, tenantId, name, address }
Stock        { id, tenantId, productId, warehouseId, qty, avgCost }
StockMovement { id, tenantId, productId, warehouseId, qty, type, refId }

Order        { id, tenantId, customerId, items, total, status,
                paymentId, cashSessionId }
OrderItem     { id, orderId, productId, qty, price, discount }
Payment       { id, tenantId, orderId, method, amount, status,
                pagbankOrderId, pagbankChargeId, paidAt,
                paymentMode (INTERNAL_WALLET|OWN_PAGBANK|OWN_OTHER) }
BillingEvent  { id, tenantId, source, payload, processed, createdAt }

PaymentMode   { tenantId, mode, credentials (encrypted), createdAt }
MerchantWallet { id, tenantId, currency, balance, blocked,
                totalReceived, totalPaidOut }
WalletTransaction { id, walletId, type, amount, balanceAfter,
                    refType, refId, createdAt }
PayoutPolicy  { tenantId, mode (AUTO_D1|MANUAL),
                minAmount, bankAccountId }
BankAccount   { id, tenantId, type, bank, agency, account,
                pixKey, pixKeyType, holderName, holderDocument }
Payout        { id, tenantId, walletId, amount, fee, netAmount,
                status, scheduledFor, processedAt,
                externalTxId, failureReason }

Cash          { id, tenantId, name }
CashSession   { id, cashId, openedBy, openedAt, closedAt, status }
CashOperation { id, sessionId, type, amount, operatorId, createdAt }

```



```

CashPhysicalClose{ id, sessionId, method, amount, diff }

ServiceOrder { id, tenantId, customerId, vehicleId, items[],
               labor[], status, warrantyTermId }
Vehicle       { id, tenantId, customerId, plate, model, brand, year }
Budget        { id, tenantId, customerId, items, total, status,
               convertedToId, convertedToType }

NcoJob        { id, tenantId, type, input, output, latencyMs,
               modelVersion, createdAt }
CatalogSyncLog { id, direction, kind, count, createdAt }

```

6 Checkout: conta interna + chave própria opcional

6.1 Princípio: fricção zero

Filosofia. O lojista entra e vende no minuto 1, sem precisar abrir conta PagBank, sem documentação, sem espera de homologação. O ERP usa a conta PagBank principal da Patria como *custodiante técnico* e mantém um *ledger interno* por lojista. Payouts automáticos transferem o saldo do lojista para a conta bancária dele cadastrada no onboarding (D+1 default; lojista pode optar por acumular). Lojista que prefira autonomia configura a chave PagBank própria a qualquer momento e o ERP passa a operar em BYOK. Taxa por transação é repassada ao lojista sem markup — a Patria não lucra na taxa.

6.2 Base técnica reaproveitada da Patria

A integração PagBank foi homologada em produção em 13/05/2026 (anexo v4, `nco.patriatechnology.com`). Já cobre:

- **PIX:** POST `/api/billing/checkout` cria order com `qr_codes`; PagBank retorna copia-cola + PNG.
- **Cartão de crédito:** tokenização client-side via SDK `pagseguro.min.js`; backend recebe apenas string `encrypted`; PAN e CVV nunca trafegam pelo backend.
- **Webhooks:** POST `/api/webhooks/pagbank` grava em `billing_events` (`processed=false`), depois processa e marca como `processed=true`. Idempotência via `reference_id = pat-<tenant>-<timestamp>`.
- **Conta principal:** `patriatechnology53@gmail.com` (PROD).

Essa stack é extraída para módulo compartilhado e usada pelo ERP em dois cenários: cobrança da mensalidade Patria (quando houver edição paga) e cobrança do cliente final do lojista no PDV/OS.

6.3 Três modos de operação por tenant

Modo	Como funciona	Quando usar
INTERNAL_WALLET (default)	Recebimento cai na conta PagBank Patria; ledger interno credita o lojista; payout D+1 (default) ou sob demanda para a conta bancária dele.	Lojista pequeno/médio que quer começar a vender hoje sem burocracia.

Modo	Como funciona	Quando usar
OWN_PAGBANK (BYOK)	Lojista configura token PagBank próprio no onboarding ou nas configurações; checkout opera diretamente na conta dele.	Lojista que já tem conta PagBank, prefere autonomia, ou quer evitar custódia.
OWN_OTHER (futuro)	Slot para outras integrações: Mercado Pago, Stone, Asaas, InfinitePay. Mesma interface, adapter por provedor.	Quando houver demanda real.

A configuração vive em `PaymentMode` por tenant; mudar de modo é operação manual com validação (não pode mudar com saldo virtual maior que zero sem saque antes).

6.4 Ledger interno (modo INTERNAL_WALLET)

A custódia técnica do saldo na conta PagBank Patria é refletida em um ledger contábil de dupla entrada simplificado:

```
MerchantWallet {
  id, tenantId, currency='BRL',
  balance,          -- saldo disponivel para payout
  blocked,          -- reservado (pre-autorizacao, contestacao pendente)
  totalReceived,    -- acumulado vitalicio recebido
  totalPaidOut,     -- acumulado vitalicio pago ao lojista
  createdAt, updatedAt
}

WalletTransaction {
  id, walletId, type, amount, balanceAfter,
  -- types:
  -- SALE_CREDIT      (venda confirmada via webhook PAID)
  -- FEE_DEBIT        (taxa PagBank repassada)
  -- PAYOUT_DEBIT     (transferencia ao lojista)
  -- PAYOUT_FEE_DEBIT (taxa do payout, se Pix tiver custo)
  -- REFUND_DEBIT     (estorno parcial/total)
  -- CHARGEBACK_DEBIT (contestacao ganha pelo cliente final)
  -- MANUAL_ADJUST    (correcao operacional, motivo obrigatorio)
  refType, refId, -- ligacao ao Order/Payment/Payout originador
  createdAt
}
```

6.5 Política de payout (modelo híbrido)

- **Default D+1.** Toda venda confirmada hoje vira payout automático no próximo dia útil para a conta bancária cadastrada. Minimiza tempo de custódia (e portanto exposição da Patria) e dá previsibilidade ao lojista.
- **Opt-in: acumular.** Lojista pode mudar para modo manual, em que o saldo acumula e ele clica em “sacar” quando quiser (mínimo configurável, ex. R\$ 50).
- **Worker** `erp-worker` roda job diário 06:00: consolida saldo, gera `Payout` pendente, dispara Pix da conta Patria para a conta do lojista, marca `PROCESSED` no retorno.

```
PayoutPolicy {
  tenantId,
  mode (AUTO_D1 | MANUAL),
  minAmount,          -- so dispara payout >= esse valor
  bankAccountId       -- destino
}
```

```

BankAccount {
  id, tenantId, type (CHECKING|SAVINGS|PIX),
  bank, agency, account, accountDigit,
  pixKey, pixKeyType (CPF|CNPJ|EMAIL|PHONE|RANDOM),
  holderName, holderDocument
}

Payout {
  id, tenantId, walletId, amount, fee, netAmount,
  status (PENDING | PROCESSING | PAID | FAILED | CANCELLED),
  scheduledFor, processedAt,
  externalTxId,          -- end-to-end ID do Pix executado
  failureReason
}

```

6.6 Taxa: repasse sem markup

A taxa cobrada pelo PagBank por transação (PIX ~0,99%, crédito 1x ~3,19%, com variações por modalidade) é **repassada integralmente ao lojista, sem markup**. Patria não lucra na taxa — isso preserva a promessa “grátis” (no sentido honesto: o lojista paga exatamente o que pagaria se tivesse conta PagBank própria). A receita Patria continua vindo da camada de IA / fornecedor, não de spread financeiro.

A FEE_DEBIT é lançada na mesma transação do SALE_CREDIT e o lojista vê o valor líquido na tela.

6.7 Fluxos suportados no PDV

- **PIX no balcão:** backend gera QR code, balcão exibe (TV, totem ou impressora térmica), cliente paga, webhook confirma, `Payment.status=PAID`, baixa de estoque, recibo, e SALE_CREDIT no MerchantWallet (modo interno) ou crédito direto na conta PagBank do lojista (BYOK).
- **Cartão presencial:** maquininha física PagBank (Order API). Status síncronico + webhook redundante.
- **Cartão remoto / link de pagamento:** URL gerada pelo backend, cliente paga sem estar fisicamente na loja (forte para material de construção, oficina e venda B2B).
- **Credenciário interno:** parcela registrada em `AccountsReceivable`, sem trânsito por PagBank (instrumento próprio do lojista, sem custódia).

6.8 Considerações on-premise

- Cliente on-premise pode operar em modo `OWN_PAGBANK` configurando `PAGBANK_TOKEN` no `.env` — checkout idêntico ao SaaS, sem dependência da Patria central.
- Cliente on-premise pode operar em `INTERNAL_WALLET` se ativar `CLOUD_SYNC_ENABLED=true` e aceitar termo específico de custódia — requer reconciliação periódica com o cloud Patria.
- Sem internet, checkout PagBank fica offline; credenciário e dinheiro continuam funcionais.

6.9 Migração futura para split nativo (PagBank Plus/Connect)

Quando o volume justificar e o PagBank Plus/Connect (ou equivalente) estiver integrado, o modo `INTERNAL_WALLET` migra para *split nativo*: o pagamento já é dividido na origem entre a conta do lojista (subaccount no PagBank/BaaS) e uma eventual conta de taxa da Patria. Custódia some, peso regulatório some, ledger continua existindo mas vira espelho do que o PagBank/BaaS reporta. Migração transparente para o lojista; nenhuma mudança de UX.

7 Sync cloud opt-in

7.1 Motivação

Sem dado agregado, vetores de receita (3) e (4) não funcionam, e a IA não melhora. Mas o on-premise por contrato *não* pode forçar nada. Solução: opt-in transparente.

7.2 Modelo do que sobe

Apenas **agregados anonimizados**. Endpoint GET `/admin/cloud-sync/preview` mostra ao cliente exatamente o que seria enviado antes de habilitar.

- Vendas agregadas por SKU/categoria/semana (sem PII, sem valor individual).
- Eventos de ruptura.
- Sinais de intenção de reposição.
- Métricas operacionais (tempo médio de OS, taxa de cancelamento).

7.3 Modelo do que desce

- Catálogo enriquecido de fornecedor (códigos, fotos, descrições, equivalências, ofertas patrocinadas).
- Atualizações de modelo NCO (novos checkpoints).
- Patches de aplicação (opt-in).

7.4 Privacidade

LGPD. Nenhum dado pessoal de cliente final do lojista (CPF, nome, telefone, endereço) sobe ao cloud em nenhum cenário. Apenas agregados estatísticos. O termo de uso explicita o que é coletado, para qual finalidade, e o cliente pode revogar a qualquer momento. Acordo de processamento de dados (DPA) padrão.

8 Fases de entrega

8.1 F1 — Núcleo comum (auth + cadastros + estoque)

Escopo. auth, installation, tenant, customer-supplier, product, warehouse, stock.
Entregáveis.

- Login/logout, recuperação de senha.
- Cadastro de empresa (Installation) e usuário inicial admin.
- CRUD de cliente/fornecedor unificado.
- CRUD de produto com hierarquia Division/Section/Group/Subgroup.
- Embedding pgvector na criação/atualização de produto e customer-supplier (Gemini no SaaS, modelo local no on-premise).
- CRUD de depósito.
- Saldo de estoque por SKU/depósito + movimentação manual (entrada, saída, ajuste).

Critério de aceite.

- Multi-tenant funcional (header `X-Tenant` isola dados em *todos* os endpoints).
- Embeddings populados; `GET /products/search?q=...` retorna por similaridade.
- Build SaaS e build on-premise (Docker Compose) ambos rodando.

8.2 F2 — Vendas + Checkout + Caixa

Escopo. order, payment, checkout, cash, promotion, price.

Entregáveis.

- Venda com múltiplos itens, descontos por item e total.
- Tabela de preços + promoções por período.
- Caixa: `CashSession` (open/partially_closed/closed), sangria, reforço, troca de operador (atômico), fechamento físico por método.
- **Checkout 100% funcional** com os três modos: `INTERNAL_WALLET` (default), `OWN_PAGBANK` (BYOK), `OWN_OTHER` (slot para futuro). PIX balcão, cartão presencial via Order API, link de pagamento remoto.
- **Wallet interno:** `MerchantWallet` + `WalletTransaction` + reconciliação automática com webhook PagBank.
- **Payout automático D+1** via worker (Pix da conta Patria para a conta bancária do lojista); opt-in de modo manual “sacar quando quiser”.
- Onboarding: lojista cadastra `BankAccount`, escolhe modo (interno é default), aceita termo de custódia se for interno.
- Repasse de taxa PagBank ao lojista sem markup, visível como `FEE_DEBIT` na timeline da wallet.
- Crediário interno (`AccountsReceivable` básico, parcelas).
- PDV web responsivo no `erp-app`.

Critério de aceite.

- Venda completa: scan/SKU → carrinho → pagamento (PIX/cartão/dinheiro) → baixa de estoque → recibo → crédito no wallet do lojista (modo interno) ou na conta dele (BYOK).
- Webhook PagBank atualiza `Payment.status`, `WalletTransaction SALE_CREDIT+FEE_DEBIT` atômicos.
- Job de payout D+1 executa, faz Pix, lança `PAYOUT_DEBIT`; lojista vê dinheiro na conta bancária no dia seguinte.
- Tenant pode alternar entre modos (com restrição: saldo zerado).
- Sessão de caixa abre, opera, fecha; relatório de fechamento bate contra contagem física.

8.3 F3 — Vertical OS (peças + serviços)

Escopo. service-order, vehicle, warranty-term, budget.

Entregáveis.

- Cadastro de veículo/equipamento por cliente, histórico de OS.
- OS com peça + mão de obra no mesmo documento, status (aberta/em andamento/aguardando peça/finalizada/entregue).
- Orçamento que converte em OS ou Order.
- Termo de garantia anexável à OS.

Critério de aceite.

- Ciclo orçamento → aprovação → execução → entrega funcional para oficina/autopeças piloto.
- Cálculo de baixa de estoque correto na finalização.
- PDF de OS gerado e impresso.

8.4 F4 — IA: NCO + sync de catálogo

Escopo. nco, catalog-sync.

Entregáveis.

- Container nco-inference empacotado, modelo TorchScript step 80k embutido, endpoints POST /partition, POST /forecast, POST /cluster.
- Worker erp-worker agendado: forecast noturno de top-N SKUs, detecção de ruptura, geração de NcoJob.
- Sugestão patrocinada de reposição na tela do PDV (com flag visual “patrocinado”).
- catalog-sync bidirecional, opt-in, com endpoint de preview.

Critério de aceite.

- Previsão de ruptura disparada com lead time suficiente para reposição em pelo menos 80% dos SKUs de giro alto da loja piloto.
- Sugestão de fornecedor aparece no carrinho com ranking explicado.
- Sync funciona ida e volta em loja com internet intermitente.

8.5 F5 — Fiscal (NF-e/NFC-e/NFS-e)

Escopo. invoice.

Entregáveis.

- Integração com homologação SEFAZ por estado (UF do tenant).
- Emissão de NF-e (B2B), NFC-e (B2C presencial), NFS-e (serviço, para OS).
- Cancelamento, carta de correção, contingência.

Crítico. Esta fase tem dependência regulatória pesada. Usar biblioteca consolidada (nfe.io, FocusNFe, eNotas) como primeira opção; implementação própria só se justificar economicamente.

8.6 F6 — Extras

Escopo. webhook, notification, logistic, integration.

- Webhooks externos para o cliente (loja envia evento para sistema próprio).
- Notificações em tempo real (websocket), in-app, e-mail.
- Frete (cálculo via integração com transportadora ou Correios) e rastreo.

9 Riscos e mitigações

Risco	Probabilidade/impacto	Mitigação
Adoção lenta por desconfiança (“ERP grátis é golpe”)	Alta / alto	Reputação Patria, casos piloto com depoimento, transparência total no modelo de receita, código aberto opcional.
Regulatório BACEN no modo <code>INTERNAL_WALLET</code> (Patria custodia saldo de terceiros na conta PagBank principal)	Média / muito alto	Quatro mitigações combinadas: (1) Payout D+1 agressivo mantém tempo médio de custódia < 24h, minimizando saldo agregado retido. (2) Limite por tenant configurável (ex. R\$ 50k de saldo máximo); acima disso forçar BYOK ou saque imediato. (3) Consulta a advogado especialista em fintech antes de operar acima de saldo agregado de R\$ 1 M ou > 100 tenants ativos no modo interno. (4) Plano explícito de migração para PagBank Plus/Connect ou BaaS (Asaas/Pagar.me/Celcoin) com sub-account emitida por IP autorizado, eliminando custódia.
Fornecedor não topa pagar até massa crítica	Alta / alto	Receita inicial não depende disso. Lojista já recebe valor real (forecast, ruptura, sugestão sem patrocínio) desde dia 1. Receita do fornecedor é camada 2, aceitar latência.
Cliente percebe ranking como “vendido”	Média / alto	Flag <i>patrocinado</i> sempre visível, ranking transparente (qualidade empírica × lance, como Google Ads), publicar metodologia.
LGPD: vazamento ou processamento indevido	Baixa / muito alto	Anonimização rigorosa no sync, opt-in explícito, DPA padrão, criptografia em trânsito e em repouso, log de acesso.

Risco	Probabilidade/impacto	Mitigação
Suporte on-premise difícil (debug remoto)	Alta / médio	Telemetria opt-in (apenas erros, sem dado), comando erp-doctor gera bundle de diagnóstico anônimo para enviar ao suporte.
NCO-HiperGNN out-of-distribution colapsa em domínio novo	Média / alto	Treinar na distribuição do cliente (custo ~ 3 dias GPU), conforme limitação documentada no paper AF; modelo fallback (heurístico) para domínios sem treino.
Concorrência (Stone, PagSeguro) replica modelo	Média / alto	Defensável por: bound matemático do NCO (paper AE), Patrícia (orquestração via Claude), tempo de mercado. Velocidade $>$ perfeição.
Bug de fiscal causa NF-e rejeitada em produção	Alta / alto	Usar provedor consolidado (FocusNFe etc.) até maturidade, fila de retentativa, alerta para diretor responsável.

10 Próximos passos imediatos

1. **Validar este plano.** Aarão revisa, ajusta escopo das fases, confirma ordem de prioridade.
2. **Definir tenant inicial de teste.** Uma loja real (idealmente material de construção ou autopeças em Boa Vista) que aceite colaborar como piloto de F1+F2.
3. **Começar F1.** Estruturar módulos **auth**, **installation**, **tenant**, **customer-supplier**, **product**, **warehouse**, **stock** no **erp-api** já criado. Schema Prisma definitivo. Primeira tela do **erp-app**.
4. **Preparar nco-inference.** Exportar TorchScript do checkpoint step 80k do GEX44, validar inferência local CPU.
5. **Reuso PagBank.** Extrair módulo **checkout** da Patria para biblioteca compartilhada (`@patria/checkout?`) ou copiar verbatim para **erp-api**. Adicionar camada **wallet + payout** acima da integração crua.
6. **Termo de custódia (modo interno).** Redigir termo específico explicando que a Patria custodia o saldo na conta PagBank principal e faz payout D+1, sem juros, sem retenção operacional, sem cobrança extra. Aceite obrigatório no onboarding de tenant que escolher modo `INTERNAL_WALLET`. Consulta rápida com advogado fintech antes de publicar.

Referências internas

Referências

- [1] Lopes, A.; Claude. *Paper AF — NCO-HiperGNN: solver neural para problemas combinatoriais de partição via álgebra hipercomplexa generalizada*, 2026.
- [2] Lopes, A.; Claude. *Paper AE — Ponte Schur-Weyl: Hochschild $\leftrightarrow$ tensorial*, 2026.

- [3] Lopes, A.; Claude. *Paper CT — Mecânica de campo médio em álgebras hipercomplexas*, 2026.
- [4] Lopes, A.; Claude. `NCO_PARTITIONING.md` — roadmap do produto NCO, 2026.
- [5] Patria Technology. *Anexo v4 — Validação PagBank em PRODUÇÃO*, 13/05/2026.
- [6] Easysync. `integrator/CLAUDE.md` — referência de arquitetura ERP de varejo maduro.
- [7] Patria Technology. `patria-api/PRODUCT.md` — visão do produto Patria.